# Embeddings, attention and the Transformer block

## Large Language Models · Module 2, lecture 1 (M2P1) · Week 2

Heudson Mirandola

Instituto de Matemática, UFRJ

November 2026

# Where we are

Week 1 left two structural defects of the $n$-gram:

1. **No sharing:** "o gato dorme" and "o cão dorme" are unrelated keys.
2. **Fixed window:** nothing older than $n - 1$ tokens exists.

Today: the neural model that answers both, one component at a time, ending with the complete decoder-only Transformer.

# One idea organizes the lecture

A Transformer alternates two operations:

one that mixes information **across positions** (attention)
and one that processes **each position on its own** (the MLP).

Attention is the only place where positions talk — and it is
a weighted average with data-dependent weights.

- **Section 1:** embeddings and Bengio's model — defect 1 solved, window kept.
- **Section 2:** attention, derived from three requirements; its four design decisions.
- **Section 3:** the block — residuals, normalization, MLP; parameters and FLOPs.
- **Section 4:** what trained attention heads actually compute.

# Objectives

By the end of the lecture you should be able to

1. explain what an embedding matrix is and why it solves the sharing problem;
2. derive scaled dot-product attention and justify each of its design decisions;
3. prove the prefix property of causal attention and say what it buys at inference;
4. write the pre-norm block and count its parameters and FLOPs;
5. explain why training is parallel and generation is not.

**Conventions:** vectors are columns; sequences are stacked as *rows* of $H \in \mathbb{R}^{n \times d}$ (PyTorch's `[batch, seq, dim]`), so linear maps act on the right: $HW$. Softmax on a matrix acts on each row.
$d$ = width, $n$ = sequence length, $V$ = vocabulary, $L$ = layers, $h$ = heads.

# Outline

1. Embeddings
2. Attention
3. The Transformer block
4. What attention learns
5. Bridge
6. Appendix: code

# One-hot vectors

A neural network needs a vector, not an integer. The trivial vectorization:

$$e_v \in \mathbb{R}^V, \qquad (e_v)_i = \begin{cases} 1 & i = v \\ 0 & \text{otherwise.} \end{cases}$$

Useless as a representation: $\|e_v - e_w\|^2 = 2$ for all $v \neq w$ — every token is equally far from every other.

But the right starting point, because **a linear map applied to $e_v$ selects a row**.

# The embedding matrix

Fix $d \ll V$ and $E \in \mathbb{R}^{V \times d}$. The embedding of token $v$:

$$x_v \;=\; E^\top e_v \;=\; \text{row } v \text{ of } E \;\in \mathbb{R}^d.$$

- A lookup table with $V$ rows of length $d$, trained by gradient descent.
- The lookup is *linear*: backprop = "add the gradient to the row used".
- `nn.Embedding` is exactly this, with the product replaced by indexing.

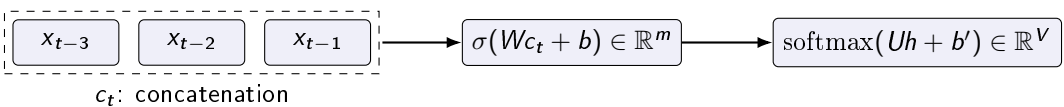
| | $V$ | $d$ | | $Vd$ |
|---|---|---|---|---|
| GPT-2 small | 50,257 | 768 | 38.6M | (31% of the model) |
| Llama 3 8B | 128,256 | 4,096 | 525M | (7%) |

# Bengio's neural language model (2003)

Keep the Markov assumption; replace the count table by a function of embeddings:

$$p_\theta(x_t \mid x_{t-n+1}, \ldots, x_{t-1}) = \mathrm{softmax}\big(U\,\sigma(Wc_t + b) + b'\big)_{x_t},$$

with $c_t = [x_{t-n+1}; \ldots; x_{t-1}] \in \mathbb{R}^{(n-1)d}$ the concatenation of the $n-1$ embeddings.




Two things change relative to the $n$-gram. Separate them.

# Change 1: parameter count

| | |
|---|---|
| trigram table | up to $V^3$ cells |
| Bengio, window $n$ | $Vd + m(n-1)d + Vm$ — *linear* in $V$ and in $n$ |

Example: $V = 20{,}000$, $d = 100$, $m = 500$, $n = 5$:

$$2\text{M} + 0.2\text{M} + 10\text{M} \quad \text{vs.} \quad V^5 = 3.2 \times 10^{21} \text{ cells.}$$

Of those $10^{21}$ cells the corpus fills a negligible fraction. The neural model has no empty cells — every parameter is used by every prediction.

# Change 2: sharing — the point

- Count table: "o gato dorme" and "o cão dorme" are unrelated keys.
- Neural model: both map to nearby points of $\mathbb{R}^{(n-1)d}$ *if* $x_{\text{gato}} \approx x_{\text{cão}}$; a smooth function then predicts alike.
- The model is *forced* to learn embeddings with that property: it is the only way to fit the data with few parameters.

**Defect 1 of the *n*-gram is solved**

Not by cleverer smoothing, but by changing the representation of the history from a discrete key to a point in a vector space, where "similar" has a meaning the model can exploit.

Bengio (2003): $-24\%$ perplexity vs. the best smoothed $n$-gram on Brown.
Defect 2 — the fixed window — untouched: $c_t$ has fixed length $(n-1)d$.

# The output layer: logits and their gradient

The last two operations survive unchanged into the Transformer:

$$z = Uh \in \mathbb{R}^V \text{ (logits)}, \qquad z_w = u_w^\top h, \qquad p_\theta(w \mid h) = \mathrm{softmax}(z)_w.$$

$U$ is the **unembedding**: one vector $u_w$ per token; the score of $w$ is a dot product with the hidden state.

Cross-entropy, differentiated with respect to the logits (notes, A.1):

$$\frac{\partial \ell}{\partial z} = p_\theta(\cdot \mid h) - e_y.$$

Predicted distribution minus one-hot target. Bounded in $[-1, 1]$; vanishes as $p_y \to 1$. Every parameter of the network receives its gradient through this vector.

# Weight tying

When $m = d$, $E \in \mathbb{R}^{V \times d}$ and $U \in \mathbb{R}^{V \times d}$ have the same shape and the same meaning — a vector per token. Set $U = E$.

- Saves $Vd$ parameters: 31% of GPT-2 small.
- Acts as a regularizer (Press & Wolf 2017; Inan et al. 2017).
- GPT-2 ties; Llama 3 8B/70B do not (saving negligible, untied fits slightly better); Llama 3.2 1B/3B tie again.

The lab's GPT (Lab 3) ties.

# Static vs. contextual embeddings

**Static** (word2vec, GloVe): train the table $E$ as the end product, with a cheap surrogate objective (predict neighbors).

- $x_{\text{king}} - x_{\text{man}} + x_{\text{woman}} \approx x_{\text{queen}}$ — explained by Levy & Goldberg (2014): skip-gram factorizes a shifted PMI matrix.
- One vector per token: `banco` (bench) and `banco` (bank) collapse.

**Contextual**: in a deep LM, $x_t$ is only the input of layer 1. The vector $h_t^{(l)}$ computed at position $t$ in layer $l$ depends on the whole prefix — a different vector for each occurrence.

The rest of the lecture: how $h_t^{(l)}$ is computed.

# The remaining problem

Replace the fixed-length $c_t$ by a summary $h_t \in \mathbb{R}^d$ of the *entire* prefix $x_1, \ldots, x_{t-1}$, under three requirements:

1. **Parameters independent of $t$** — otherwise a lookup table over histories.
2. **Parallel over $t = 1, \ldots, n$** — otherwise training on $10^{12}$ tokens is impractical (this is what killed RNNs).
3. **Retrieve specific information from specific positions** — not a blurred average (copying, agreement, reference).

**Attention satisfies the three.**

# Outline

1. Embeddings
2. Attention
3. The Transformer block
4. What attention learns
5. Bridge
6. Appendix: code

# Averaging as a starting point

Simplest fixed-size summary of $v_1, \ldots, v_t$: the mean $h_t = \frac{1}{t} \sum_{s \le t} v_s$.
No parameters, trivially parallel, useless: all past positions alike.

Next simplest: a **weighted** average with data-dependent weights,

$$h_t = \sum_{s=1}^{t} a_{ts} \, v_s, \qquad a_{ts} \ge 0, \quad \sum_{s \le t} a_{ts} = 1.$$

The row $(a_{t1}, \ldots, a_{tt})$ is the **attention pattern** of position $t$.
The whole of attention is a choice of how to compute it from the data.

# Scores and softmax

Let the weight depend on how *relevant* $s$ is to $t$:

$$a_{ts} = \frac{\exp \operatorname{score}(h_t, h_s)}{\sum_{s' \le t} \exp \operatorname{score}(h_t, h_{s'})}.$$

The softmax does two jobs:

- makes the weights positive and normalized;
- because exp amplifies differences, it can put almost all weight on one position when one score dominates — requirement 3 — while staying differentiable, unlike arg max.

# Queries, keys, values

Vaswani et al. (2017): the score is a dot product between two *linear projections*,

$$q_t = W_Q^\top h_t, \qquad k_s = W_K^\top h_s, \qquad \mathrm{score}(h_t, h_s) = \frac{q_t^\top k_s}{\sqrt{d_k}},$$

$W_Q, W_K \in \mathbb{R}^{d \times d_k}$. The vector averaged is a third projection, $v_s = W_V^\top h_s$, $W_V \in \mathbb{R}^{d \times d_v}$.

| | |
|---|---|
| **query** $q_t$ | what position $t$ is looking for |
| **key** $k_s$ | what position $s$ offers |
| **value** $v_s$ | what position $s$ contributes once attended |

# Three questions a careful reader asks

**Why not $h_t^\top h_s$ directly?** The dot product is symmetric; relevance is not (a verb looks for its subject). With $W_Q, W_K$ the score is $h_t^\top (W_Q W_K^\top) h_s$: a general bilinear form of rank $\leq d_k$, at $2dd_k$ parameters instead of $d^2$.

**Why separate values from keys?** "What makes $s$ worth attending to" and "what $s$ contributes" are different information. Key: *I am a noun in subject position*. Value: *I am plural*.

**Why a dot product and not an MLP on $(h_t, h_s)$?** Bahdanau et al. (2015) used $w^\top \tanh(W_1 h_t + W_2 h_s)$. The dot product, for all $(t, s)$ at once, is *one* matrix product $QK^\top$ — the most optimized operation on a GPU. Requirement 2 decides.

# The soft-dictionary reading

| | Python `dict` | attention |
|---|---|---|
| match | key = query, exactly | $q^\top k$, graded |
| result | the one value | convex combination of all values |
| one key dominates | — | nearly a hard lookup |
| several match equally | error | their average |

Everything attention can do is a **differentiable, parallel, associative memory lookup**.

# The $\sqrt{d_k}$ scaling: a one-line variance calculation

**Proposition**

If $q, k \in \mathbb{R}^{d_k}$ have independent components with mean 0 and variance 1, then $\mathbb{E}[q^\top k] = 0$ and $\mathrm{Var}[q^\top k] = d_k$.

*Proof.* $q^\top k = \sum_i q_i k_i$; each term has mean 0 and variance $\mathbb{E}[q_i^2]\mathbb{E}[k_i^2] = 1$; the $d_k$ terms are independent. □

At initialization the raw scores have standard deviation $\sqrt{d_k}$: $\approx 8$ for $d_k = 64$, 32 for $d_k = 1024$. A softmax of entries differing by tens is a one-hot vector.

# Why a saturated softmax is fatal

Jacobian of the softmax (notes, A.2):

$$\frac{\partial a_i}{\partial s_j} = a_i\,(\delta_{ij} - a_j), \qquad J^\top g = a \odot \big(g - \langle a, g\rangle 1\big).$$

If $a \approx e_k$: for $j \neq k$ the factor $a_j \approx 0$; for $j = k$, $g_k - \langle a, g\rangle \approx 0$. **Every coordinate of the gradient vanishes.**

- No gradient reaches $W_Q, W_K$: the pattern never leaves its random initial state.
- Dividing by $\sqrt{d_k}$ brings the scores back to unit variance — the responsive regime.

*General lesson: the scale of every intermediate quantity must be controlled — by normalization, initialization, or constants like this one. Module 3 returns to it.*

# The causal mask

Position $t$ predicts $x_{t+1}$; if it could read $x_{t+1}$ it would copy it and learn nothing. Impose the constraint before the softmax:

$$\tilde{s}_{ts} = \begin{cases} s_{ts} & s \le t \\ -\infty & s > t \end{cases} \qquad \Longrightarrow \qquad e^{-\infty} = 0 \text{ in numerator and denominator.}$$

- Exactly the sum over $s \le t$ of the first slide — but computed for all positions at once by one matrix operation.
- In code: a lower-triangular matrix of ones; $-\infty$ is a large negative float.
- One forward pass yields all $n$ conditional distributions: **teacher forcing**.

# The prefix property

**Proposition**

A network built from position-wise maps and causally masked attention has the property that its output at position $t$ is a function of $x_1, \ldots, x_t$ only. Consequently, its output at $t$ on $x_{1:n}$ equals its output at $t$ on the prefix $x_{1:t}$.

*Proof.* Induction over layers; a masked layer at $t$ reads only $q_t$ and $k_s, v_s$ for $s \leq t$, themselves functions of $x_{1:t}$. Truncating after $t$ changes nothing. □

The second claim buys two things:

- **generation is consistent with training**: fed its own prefix, the model computes exactly the distributions it was trained on;
- **the KV cache is valid** (Module 7): past keys and values never change.

*No mask = bidirectional (BERT). Queries from one sequence, keys/values from another = cross-attention. Week 3.*

# Matrix form

Stack the $n$ inputs as rows of $H \in \mathbb{R}^{n \times d}$:

$$Q = HW_Q, \qquad K = HW_K, \qquad V = HW_V,$$

$$\boxed{\mathrm{Attention}(Q, K, V) = \mathrm{softmax}\Big(\frac{QK^\top}{\sqrt{d_k}} + M\Big)\, V \;\in \mathbb{R}^{n \times d_v}}$$

$M_{ts} = 0$ for $s \le t$, $-\infty$ otherwise; softmax along each row.

$A = \mathrm{softmax}(QK^\top/\sqrt{d_k} + M) \in \mathbb{R}^{n \times n}$ is the **attention matrix**:
lower triangular, each row a probability distribution over $\{1, \ldots, t\}$.

# Shapes — every bug in Lab 2 will be a shape bug

| operation | shapes | result |
|---|---|---|
| projections | $(n \times d)(d \times d_k)$ | $n \times d_k$ |
| scores $QK^\top$ | $(n \times d_k)(d_k \times n)$ | $n \times n$ |
| output $AV$ | $(n \times n)(n \times d_v)$ | $n \times d_v$ |

With a batch dimension $B$ in front (`[B, n, d]`), PyTorch broadcasts the matrix products over it. With heads: `[B, h, n, d/h]`.

# Multi-head attention: why

One attention layer gives each position *one* pattern over the past.

A position may need several things at once: the previous token, the subject of the sentence, an open bracket. One softmax cannot concentrate on two positions without blurring both.

Fix: run $h$ attentions in parallel, each with its own projections and reduced dimension $d_k = d_v = d/h$, and concatenate:

$$\mathrm{head}_i = \mathrm{Attention}(HW_Q^{(i)}, HW_K^{(i)}, HW_V^{(i)}), \qquad \mathrm{MHA}(H) = [\mathrm{head}_1 \mid \cdots \mid \mathrm{head}_h]\, W_O.$$

Each head: its own $A^{(i)} \in \mathbb{R}^{n \times n}$, its own $d/h$-dimensional subspace to read and write. $W_O \in \mathbb{R}^{d \times d}$ mixes the heads back into the stream.

# Multi-head attention: cost and layout

$$\#\text{params}(\mathrm{MHA}) = \underbrace{3d^2}_{W_Q, W_K, W_V} + \underbrace{d^2}_{W_O} = 4d^2 \qquad \text{— independent of } h.$$

- The $h$ heads' projections side by side form $W_Q \in \mathbb{R}^{d \times d}$ (same for $K$, $V$): in code, *one* $d \times 3d$ product and a reshape. Never a loop over heads.
- $h$ trades the dimension of each head against the number of patterns.

| | $d$ | $h$ | $d/h$ |
|---|---|---|---|
| GPT-2 small | 768 | 12 | 64 |
| GPT-3 175B | 12,288 | 96 | 128 |
| Llama 3 (all sizes) | 4,096–16,384 | 32–128 | 128 |

*Per-head dimension stays fixed; the number of heads grows with $d$. GQA: Week 3.*

# What attention does not know: position

**Proposition (permutation equivariance)**

For a permutation matrix $P$ and *unmasked* attention,
$\mathrm{Attention}(PH) = P\,\mathrm{Attention}(H)$.

*Proof.* $Q' = PQ$, $K' = PK$, $V' = PV$; $S' = PSP^\top$; row-wise softmax commutes with simultaneous row/column permutation; $A'V' = PAP^\top PV = PAV$. $\square$

Position-wise maps are equivariant too, so the whole network processes a *set*:
"o gato mordeu o cão" $\equiv$ "o cão mordeu o gato", up to permutation of outputs.

**Something has to break the symmetry.**

# Positional information, option 1: learned absolute

GPT-2, BERT: a second table $P \in \mathbb{R}^{n_{\max} \times d}$; feed $x_t + p_t$ to layer 1.

- Simple, works.
- No row $p_{n_{\max}+1}$: cannot process longer sequences at all.
- Must learn separately, for each pair of positions, how far apart they are.

# Option 2: sinusoidal (Vaswani et al. 2017)

A fixed table, a bank of $d/2$ sinusoids with geometrically spaced frequencies:

$$p_{t,2i} = \sin(\omega_i t), \qquad p_{t,2i+1} = \cos(\omega_i t), \qquad \omega_i = 10000^{-2i/d}.$$

Wavelengths from $2\pi$ to $2\pi \cdot 10^4$ — a clock with $d/2$ dials of increasing period.

The property that learned tables lack (notes, A.3): for every $\omega_i$ and offset $k$,

$$\begin{pmatrix} \sin \omega_i(t+k) \\ \cos \omega_i(t+k) \end{pmatrix} = \underbrace{\begin{pmatrix} \cos \omega_i k & \sin \omega_i k \\ -\sin \omega_i k & \cos \omega_i k \end{pmatrix}}_{\text{rotation by } \omega_i k, \text{ independent of } t} \begin{pmatrix} \sin \omega_i t \\ \cos \omega_i t \end{pmatrix}.$$

Position $t + k$ is a fixed linear function of position $t$.

# From sinusoids to RoPE (preview of Week 3)

- Sinusoids are *added* to the embeddings and then multiplied by $W_Q, W_K$: $q_t^\top k_s$ contains cross terms between content and position. The relative property is weakened.
- **RoPE** applies the rotation to $q$ and $k$ themselves, so that $q_t^\top k_s$ depends on $t - s$ exactly. The $2 \times 2$ rotation above is literally its building block.

*Remark: the causal mask already breaks the symmetry partially (position t sees t keys). Haviv et al. (2022): decoders with no positional encoding at all learn competitive LMs. Encodings make position easy, not possible.*

# The cost of attention: FLOPs

Per head, $QK^\top$ and $AV$ each cost $n \cdot n \cdot d_k$ multiply-adds; over $h$ heads with $hd_k = d$ (a multiply-add = 2 FLOPs):

$$\text{FLOPs}(QK^\top) + \text{FLOPs}(AV) \approx 4n^2 d.$$

Projections: $8nd^2$. MLP of the block (Section 3): $16nd^2$. Per layer:

$$\boxed{24\, nd^2 \;+\; 4\, n^2 d}$$

Quadratic term dominates when $4n^2 d > 24nd^2$, i.e. $n > 6d$.

- GPT-2 small ($d = 768$): crossover at $n \approx 4600$, beyond its context of 1024; at $n = 1024$ the $n^2$ term is $\sim 18\%$ of the total.
- $d = 4096$: crossover at $n \approx 25{,}000$.

"Attention is quadratic and therefore dominates": false for short contexts, true for long ones. Where the line is depends on $d$.

# The cost of attention: memory is the immediate problem

A naive implementation materializes $A \in \mathbb{R}^{h \times n \times n}$ per layer and per sequence, and keeps it for the backward pass.

| $h = 12$, fp32 | $n = 1024$ | $n = 8192$ |
|---|---|---|
| per layer, per sequence | 48 MB | 3 GB |

At $n = 8192$, one layer's attention matrix for one sequence exceeds the parameters of the whole model. Lab 2 measures this growth.

**FlashAttention** (Dao et al. 2022; Week 4): same output, never materializes $A$ — tiles the softmax and normalizes on the fly. $O(n^2)$ time stays; $O(n^2)$ memory goes; avoiding the trip to GPU memory makes it much faster in practice.

*Hook for Modules 3 and 7: at inference with a KV cache, one new token costs $O(nd)$ in attention; the problem becomes the cache's memory, $2Lnd$ numbers per sequence.*

# Outline

1. Embeddings
2. Attention
3. The Transformer block
4. What attention learns
5. Bridge
6. Appendix: code

# Attention mixes positions; everything else is position-wise

- Most parameters and most FLOPs are in the "everything else".
- Four components; taken in the order they were *understood*, not the order in the code.

| component | acts on | role |
|---|---|---|
| residual connection | the stream | identity path; makes depth trainable |
| layer normalization | each position | controls scale |
| MLP | each position | processes what attention gathered |
| attention | across positions | the only coupling between positions |

# The residual stream

Each component $f$ is applied as $h \leftarrow h + f(h)$, not $h \leftarrow f(h)$ (He et al. 2016). Unrolled over the $2L$ sublayers:

$$h_t^{(\text{out})} = x_t + p_t + \sum_{l=1}^{2L} f_l\big(h^{(l-1)}\big)_t.$$

The vector at position $t$ is a running sum — the residual stream — to which every sublayer *adds* a contribution. No sublayer transforms the stream; each one reads from it and writes into it.

Consequence: the stream has fixed width $d$ throughout. $W_O$ and the MLP's $W_2$ exist to map back into it.

# Why residuals matter, twice

**Optimization.** The Jacobian of $h \mapsto h + f(h)$ is $I + \partial f/\partial h$. The gradient to any early layer contains a term that passes through all the identities, undamped, however badly conditioned the $f_l$ are. Networks of this depth were not trainable before.

**Interpretation.** Because the stream is a sum, one can ask what each sublayer *added* — and, via the unembedding, which logits each attention head or MLP pushed. Section 4 uses this.

# Layer normalization

Standardize each position's vector across its $d$ coordinates (Ba, Kiros & Hinton 2016):

$$\mathrm{LN}(h) = \gamma \odot \frac{h - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta, \qquad \mu = \frac{1}{d}\sum_i h_i, \quad \sigma^2 = \frac{1}{d}\sum_i (h_i - \mu)^2.$$

- Statistics *per token*, over features: independent of batch and sequence length (unlike batch norm — hence LN for sequences).
- Scale invariant: $\mathrm{LN}(\alpha h) = \mathrm{LN}(h)$ for $\alpha > 0$.
- Gradient through it scales as $1/\sigma$ (notes, A.4): larger activations get smaller gradients — self-stabilizing.

# Pre-norm vs. post-norm

| | update | identity path |
|---|---|---|
| post-norm (Vaswani 2017) | $h \leftarrow \mathrm{LN}(h + f(h))$ | interrupted $2L$ times |
| pre-norm (GPT-2 and after) | $h \leftarrow h + f(\mathrm{LN}(h))$ | unbroken |

Looks cosmetic; is not. Xiong et al. (2020): post-norm needs learning-rate warmup to train at all; pre-norm does not.

Default in these notes and in the labs: **pre-norm**.

*Week 3: RMSNorm — drop $\mu$ and $\beta$, normalize by the root mean square.*

# The position-wise MLP

Same two-layer network at every position:

$$\mathrm{MLP}(h) = W_2\,\sigma(W_1 h + b_1) + b_2, \qquad W_1 \in \mathbb{R}^{4d\times d}, \quad W_2 \in \mathbb{R}^{d\times 4d}.$$

Hidden width $4d$ by convention; $\sigma = \mathrm{GELU}$, $x\,\Phi(x)$, a smooth ReLU.

$$\#\mathrm{params}(\mathrm{MLP}) = 4d \cdot d + d \cdot 4d = 8d^2 \qquad \text{— twice attention.}$$

Two thirds of the non-embedding parameters are in the MLPs; two thirds of the FLOPs too ($16nd^2$ vs. $8nd^2$ for the attention projections).
Applied independently per position = a $1 \times 1$ convolution; cost linear in $n$.

# The MLP as a key–value memory

Rows of $W_1$: keys $\kappa_1, \ldots, \kappa_{4d} \in \mathbb{R}^d$. Columns of $W_2$: values $\nu_1, \ldots, \nu_{4d} \in \mathbb{R}^d$. Then

$$\mathrm{MLP}(h) = \sum_{j=1}^{4d} \sigma\left(\kappa_j^\top h + b_{1,j}\right) \nu_j.$$

Each hidden unit tests the input against a key and, if it matches, adds its value to the stream (Geva et al. 2021).

| | keys | looks up |
|---|---|---|
| attention | computed from the context | the current sequence |
| MLP | learned, input-independent | patterns seen in training |

*Factual knowledge edited in a model (Meng et al. 2022) is found in middle-layer MLPs.*

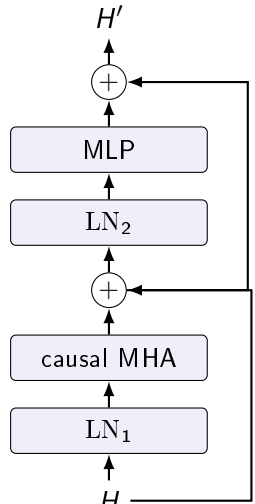
# The block

Pre-norm, decoder-only:

$$H \leftarrow H + \mathrm{MHA}\big(\mathrm{LN}_1(H)\big)$$
$$H \leftarrow H + \mathrm{MLP}\big(\mathrm{LN}_2(H)\big)$$

MHA causally masked; MLP row by row.

Per block: $4d^2 + 8d^2 = 12d^2$ weights, $+ O(d)$.



# The full model

Inputs $x_{1:n}$:

1. **Embed:** $H^{(0)} = E[x_{1:n}] + P[1{:}n] \in \mathbb{R}^{n \times d}$.
2. **Blocks:** $H^{(l)} = \mathrm{Block}_l(H^{(l-1)})$, $l = 1, \ldots, L$.
3. **Final norm, unembed:** $Z = \mathrm{LN}_f(H^{(L)})\, U^\top \in \mathbb{R}^{n \times V}$; row $t$ = logits for $x_{t+1}$.
4. **Loss:** $\mathcal{L} = -\frac{1}{n} \sum_t \log \mathrm{softmax}(Z_t)_{x_{t+1}}$ — Week 1's cross-entropy, averaged over positions.

| | $V$ | $n_{\max}$ | $d$ | $L$ | $h$ |
|---|---|---|---|---|---|
| GPT-2 small | 50,257 | 1,024 | 768 | 12 | 12 |
| GPT-2 XL | 50,257 | 1,024 | 1,600 | 48 | 25 |
| Llama 3 8B | 128,256 | 8,192 | 4,096 | 32 | 32 |

*The reference implementation (notes, Appendix B) is 70 lines of PyTorch.*

# Counting parameters

$$N \approx \underbrace{12\,L\,d^2}_{\text{blocks}} + \underbrace{Vd}_{\text{embedding}} + \underbrace{n_{\max} d}_{\text{positions}} \qquad \text{(unembedding tied)}$$

| | $12Ld^2$ | $Vd$ | $n_{\max}d$ | total | published |
|---|---|---|---|---|---|
| GPT-2 small | 84.9M | 38.6M | 0.8M | 124.3M | 124M |
| GPT-2 XL | 1,475M | 80.4M | 1.6M | 1,557M | 1.5B |

Exact enough to plan a model with.

# Two readings of the formula

**Embedding share.** $Vd$ is linear in $d$, $12Ld^2$ quadratic: the embedding is 31% of GPT-2 small, 5% of GPT-2 XL, $< 2\%$ of Llama 3 70B.
Scaling-law papers report **non-embedding** parameters, $12Ld^2$, for this reason.

**Depth vs. width.** $Ld^2$: doubling $d$ costs $4\times$, doubling $L$ costs $2\times$. The aspect ratio $d/L$ of trained models stays in a narrow band (64 for GPT-2 small, 128 for Llama 3 8B): near-optimal over a wide range for a fixed budget (Kaplan et al. 2020).

# Counting FLOPs: two per parameter per token

Forward pass, summing Section 2's per-layer count over layers, adding the unembedding, ignoring the $n^2$ term for $n < 6d$:

$$\text{FLOPs}_{\text{fwd}} \approx 24Lnd^2 + 2nVd = 2n\,(12Ld^2 + Vd) \approx 2\,n\,N.$$

Every weight is used once per token in a matrix–vector product: one multiply, one add.

Backward $\approx 2\times$ forward (gradient w.r.t. activations, gradient w.r.t. weights — notes, A.5), so training $N$ parameters on $D$ tokens costs

$$\boxed{C \approx 6\,N\,D}$$

GPT-2 small on $10^{10}$ tokens: $7.4 \times 10^{18}$ FLOPs $\approx$ 5 h of one H100 at 40% utilization.
*Lab 4 measures the utilization; Week 5 uses $C = 6ND$ as the scaling-law budget.*

# Training is parallel

The Transformer did not displace the LSTM by being more expressive. It displaced it by the shape of its computation.

- One forward pass on $x_{1:n}$ gives $p_\theta(\cdot \mid x_{1:t})$ for every $t$ at once (prefix property). Loss: row $t$ vs. $x_{t+1}$ — the input shifted by one.
- No sequential dependency between positions: the sequence dimension is as parallel as the batch dimension.
- An RNN computes $h_t$ from $h_{t-1}$: wall-clock $\Omega(n)$ per sequence, regardless of processors.

*Teacher forcing's known cost — exposure bias: at generation the model conditions on its own outputs, never seen in training. Module 6 is partly about correcting this.*

# Generation is sequential

$x_{n+1}$ must be drawn before $x_{n+2}$ can be conditioned on it.

Naively: a full forward pass per token; $m$ tokens after a prompt of length $n$ cost

$$\sum_{t=n}^{n+m} O(td^2 + t^2 d) \qquad \text{— cubic in the length.}$$

Rescue: the prefix property. Past keys and values never change, so **cache them**; each new token costs one position's projections plus one query against the cache, $O(d^2 + nd)$ per layer.

That is the **KV cache**. Its memory, $2Lnd$ numbers per sequence, is what limits batch size and context at serving time (Module 7).

# Outline

1. Embeddings

2. Attention

3. The Transformer block

4. What attention learns

5. Bridge

6. Appendix: code

# Looking inside

The stream is a sum; each head writes through its own $W_O^{(i)}$. So heads can be inspected one at a time: by their pattern $A^{(i)}$ and by what they add.

**Positional heads** (early layers of GPT-2): attend almost exclusively to the previous token (sharp sub-diagonal in $A$), to the first token, or to the token itself.

A previous-token head builds bigram-like features: after it, position $t$ carries $x_{t-1}$ as well as $x_t$. One such head per layer: $L$ layers assemble features of the last $L$ tokens — **the $n$-gram window, reconstructed**.

# Induction heads (Elhage et al. 2021; Olsson et al. 2022)

Sequence: `... A B ... A ?`

An **induction head** attends from the current `A` to the token *after* the previous `A` — that is, to `B` — and copies it into the prediction.

Two heads, two layers:

1. a previous-token head writes "my predecessor was `A`" into the stream at the old `B`;
2. a later head with query "I am `A`" and keys reading "my predecessor was `A`" attends there and copies the token identity out.

No $n$-gram can do this: arbitrary distance, and `AB` need never have occurred in training. The simplest form of **in-context learning**; the ability jumps in training exactly when induction heads form.

# Attention sinks, and the moral

**Attention sinks** (Xiao et al. 2023). In most trained models a large share of every head's mass goes to the first token, whatever it is. A softmax must put its mass somewhere even when nothing is relevant; the first token, visible to all, is the dump. Evict it from the KV cache and the model degrades sharply.

## The moral for this lecture

The components of Sections 2 and 3 are not "a function approximator". They have identifiable algorithmic roles — lookup by content, copying, composition across layers — and these are exactly the operations the $n$-gram lacked.

# Outline

1. Embeddings
2. Attention
3. The Transformer block
4. What attention learns
5. Bridge
6. Appendix: code

# Lab 2 (M2P2), this week

In plain PyTorch:

1. Scaled dot-product attention with a causal mask, from the boxed formula; check against `F.scaled_dot_product_attention`.
2. Multi-head causal self-attention in the one-product-plus-reshape layout; check numerically against `nn.MultiheadAttention` with the same weights.
3. The pre-norm block and a parameter counter that reproduces $12Ld^2 + Vd + n_{\max}d$ for GPT-2's four sizes.
4. Memory and time of the attention layer vs. $n$: confirm $n^2$, find where the naive implementation runs out of memory.

Lab 3 stacks the block into a $\sim$10M-parameter GPT, trained on the Lab 1 corpus, evaluated in bits per byte against the $n$-gram baseline.

# Week 3

Replace the pieces of the block, one by one, with their current versions:

| today | Week 3 | motivation |
|---|---|---|
| LayerNorm | RMSNorm | cheaper, same effect |
| GELU MLP, $8d^2$ | SwiGLU, three matrices | quality per parameter |
| learned positions | RoPE | relative positions, extrapolation |
| $h$ KV heads | GQA: fewer KV heads | KV-cache memory |
| biases | none | simplicity |

Then the three families (encoder / decoder / encoder–decoder: a question about the mask and the objective, not the block), and the escapes from $n^2$ (linear attention, SSMs) and from $12Ld^2$ (mixture of experts).

# Exercises (due with Lab 2) — see the notes for the full statements

1. Scaling with non-unit variances; why $\sqrt{d_k}$ is kept fixed anyway.
2. Softmax Jacobian; $J^\top g \to 0$ as $a \to$ one-hot.
3. Prefix property: which of LN, batch norm, centered/shifted convolutions, dropout break it?
4. Equivariance: complete the proof; show masked attention is not equivariant; how a causal model recovers position without encodings.
5. Parameter and FLOP counts for GPT-2 medium/large and Llama 3 8B; the $n^2$ crossover.
6. A bigram model in one attention head; a trigram with a previous-token head + MLP.
7. Memory of attention matrices for $L = h = 32$, batch 16, bf16: when is it 80 GB?
8. Discussion: parallelism vs. expressivity — Transformer vs. RNN.

# Outline

1. Embeddings
2. Attention
3. The Transformer block
4. What attention learns
5. Bridge
6. Appendix: code

# Causal self-attention (notes, B.1) — the core

```
B, n, d = x.shape
q, k, v = self.qkv(x).split(d, dim=-1)          # one d x 3d product
q = q.view(B, n, self.h, self.dk).transpose(1, 2)   # [B, h, n, dk]
k = k.view(B, n, self.h, self.dk).transpose(1, 2)
v = v.view(B, n, self.h, self.dk).transpose(1, 2)

s = (q @ k.transpose(-2, -1)) / math.sqrt(self.dk)  # [B, h, n, n]
s = s.masked_fill(~self.mask[:, :, :n, :n], float("-inf"))
a = F.softmax(s, dim=-1)                            # rows sum to 1
y = a @ v                                           # [B, h, n, dk]
y = y.transpose(1, 2).contiguous().view(B, n, d)    # concatenate heads
return self.proj(y)                                 # W_O
```

The four lines from `s = ...` to `y = a @ v` are the boxed formula. In production: `F.scaled_dot_product_attention(q, k, v, is_causal=True)`.

# Block and model (notes, B.2)

```
class Block(nn.Module):
    def forward(self, x):
        x = x + self.attn(self.ln1(x))       # mixes positions
        x = x + self.mlp(self.ln2(x))        # position-wise
        return x

class GPT(nn.Module):
    def forward(self, idx, targets=None):   # idx: [B, n]; targets: idx shifted by one
        pos = torch.arange(idx.size(1), device=idx.device)
        x = self.tok_emb(idx) + self.pos_emb(pos)
        for block in self.blocks:
            x = block(x)
        logits = self.head(self.ln_f(x))    # [B, n, V]; head.weight is tok_emb.weight
        if targets is None:
            return logits, None
        return logits, F.cross_entropy(logits.flatten(0, 1), targets.flatten())
```

GPT(50257, 1024, 768, 12, 12) has 124.4M parameters. The structure of these 20 lines has been stable since 2019.

# Proofs in the notes

- **A.1** Gradient of the cross-entropy w.r.t. the logits: $p - e_y$.
- **A.2** Jacobian of the softmax; vanishing under saturation.
- **A.3** Sinusoidal positions: the shift by $k$ is a $t$-independent rotation; $p_t^\top p_{t+k}$ depends on $k$ only.
- **A.4** Scale invariance of LN and its Jacobian $\frac{1}{\sigma}\big(I - \frac{1}{d}11^\top - \frac{1}{d}\hat{h}\hat{h}^\top\big)$; effective learning rate $\eta/\|W\|^2$.
- **A.5** Why backward $= 2\times$ forward: $GW^\top$ and $X^\top G$.